# Boolean Index Alignment: Deep Dives and Claim–Evidence Matrix

Manuel Coleman · Independent-review revision · November 5, 2026

Nine companion essays and the current 47-claim map. Definitions and examples are proposed or conditional; no LLM efficacy evaluation is reported. The archived review and source errata are in the package audit and sources folders.

---

# 01 — Index Foundations: Typed Indices, Registries, Lineage, and Scale Trees

*Backs manuscript §§1.1, 2.1, 3.1–3.2, 3.6 and Appendix C.*

## 1.1 The problem the index abstraction solves

A sparse language-model system does not produce text from a single undifferentiated computation. At each step it makes discrete selections: a top-k gate chooses experts inside a layer (Shazeer et al., 2017; Fedus et al., 2022; Jiang et al., 2024); an orchestrator chooses an agent (Wu et al., 2023; Fourney et al., 2024); a retriever chooses passages (Lewis et al., 2020); a planner chooses a tool (Schick et al., 2023; Yao et al., 2023); a memory manager chooses a path (Packer et al., 2023). Each selection is an identifier drawn from a finite set. Coleman (2026a, §3.1) named these identifiers *indices* and observed that a large class of production failures is a mismatch between indices rather than a defect of fluency: the citation that names a document never retrieved, the tool call that uses a deregistered version, the summary that silently drops a committed fact.

The value of the abstraction is that indices are *observable*. Hidden activations are not certifiable objects; an expert ID in a router log, a document hash in a retrieval record, and a commit record in a ledger are. This finite verifier requires an explicitly represented, enumerable obligation universe. Typed indices supply one such representation for the exposed trace; they are not a necessary condition for every meaningful boolean proposition or every possible verifier. Whether this abstraction captures a task's relevant failures remains an empirical coverage question.

## 1.2 Why every index must be typed and versioned

Definition 1 writes an index as `ι = (type, name, epoch, version)`. Each component removes a specific ambiguity that otherwise breaks a checker.

**Type.** Coleman (2026a) already used a disjoint union `ℐ_expert ∪ ℐ_agent ∪ ℐ_mem ∪ ℐ_tool ∪ ℐ_doc`. The continuation extends it with `claim`, `correction`, `source`, and `obligation`, because those are the objects corrections and continuity act on. Disjointness matters operationally: a string such as `search` can be a tool, an agent persona, and a memory namespace at once. A checker that compares untyped names can certify that "the cited `search` was active" when the active `search` was a tool and the cited one a document. Typing converts that silent category error into a failed cast (`⊥`) in Definition 3.

**Epoch.** A registry changes. Tools are deprecated, experts are retrained, policies are revised. The epoch names the registry state against which liveness and permission are evaluated. Without it, `Live(ι)` is ill-posed: live *when*? Trace E in the manuscript (stale tool authority) depends entirely on this field — the approval was genuine under `r7` and obsolete under `r8`.

**Version.** The version distinguishes contents or interface of the resource itself. A document revised from version 2 to 3 (Trace G) keeps its name; claims supported by version 2 may be unsupported by version 3. Version is the hook by which a correction finds its dependents.

The rule "an index name without a type is not a well-formed certificate field" is therefore a parsing rule with semantic consequences: it forces every downstream predicate to operate on unambiguous objects.

## 1.3 Registry entries and the separation of liveness from permission

A registry entry `Reg(ι) = (type, capabilities, input_schema, output_schema, authority_scope, validity_interval, digest)` supports two predicates the manuscript keeps apart:

- `Live_k(ι)`: the entry exists at scale `k`, its validity interval covers the relevant logical time, and its digest matches the observed artifact.
- `Permitted_k(ι, a)`: policy `Π_k` permits action `a` on that entry for the authenticated principal.

The distinction is the content of Trace B. A registered, live, in-budget expert can be activated for a task it is not permitted to perform. Registry membership answers "does this thing exist and is it the thing I think it is?"; policy answers "may this principal use it for this?". Conflating them creates an authorization failure when a tool's presence in the tool list is treated as license to call it. No comparative prevalence claim is established here. The digest field closes a third gap — substitution — by binding the name to the artifact actually executed.

The policy is declared an *input* to the audit, "not something inferred from successful execution." This is a deliberate refusal of a tempting inference. A call that succeeded shows that the environment accepted it; it does not show that it was permitted. Many environments accept calls they should not.

## 1.4 Activation records and why gate weights are telemetry

Definition 2's activation record `r = (ι, parent, gate, input_digest, output_digest, principal, event_id)` records the fact of selection. The manuscript states that "a nonzero gate does not imply permission, authority, support, or correctness." The reason is mechanistic. In a softmax gate `p_i(x) = exp(h_i)/Σ_j exp(h_j)` followed by top-k (Coleman 2026a, §3.1, following Shazeer et al. 2017), the gate value measures the router's learned affinity between an input and an expert. It is trained for loss reduction, often with an auxiliary load-balancing term (Lepikhin et al., 2020; Fedus et al., 2022). Nothing in that objective encodes authorization or factual support. A high gate on a wrong expert and a low gate on the right one are both ordinary. Hence `A_route` never reads gate magnitude as quality; gates enter only as telemetry and, where a budget is declared, through `Budget(r)`.

## 1.5 Morphisms and lineage: the pointer discipline generalized

Pointer Networks (Vinyals et al., 2015) make a model's output an index into its input, which guarantees by construction that the output refers to something present. Definition 3 generalizes that guarantee across stores. A typed morphism `μ: ℐ_a → ℐ_b ∪ {⊥}` records a legal cast — document to memory path, tool output to ledger subject. A lineage for `c` is a witnessed path `ι_0 → … → c` with `ι_0` in the visible activation history `R_{≤t}`.

Three properties follow directly from the definition and are used throughout the manuscript:

1. **Existence is not lineage.** "If no path exists, `c` is unaligned with the activation history even when its display name is real in some registry." A citation to a real document that was never activated in this run fails. This is the "index echo" falsifier (§11.9, item 8).
2. **Lineage is not support.** A path shows that evidence was visible, not that it entails the claim. The generation predicate (deep dive 02 and §4.4) therefore adds `Fidelity` separately.
3. **Lineage is finite and checkable.** Given a finite trace, deciding whether a witnessed path exists is graph reachability, which is linear in nodes and edges. This is the basis of the verification bound in Proposition 10.

## 1.6 Content-bound receipts: what a hash does and does not show

Definition 4 introduces the receipt `R(u) = (digest, version, quoted_span, span_offset, verifier_id, intake_event, vector_clock, scope)`. The key analytical point is the difference between *binding* and *reading*. Given candidate byte strings, matching SHA-256 digests provide an integrity check under the cryptographic assumptions. A party can copy a digest without possessing the source bytes. Neither possession of a digest nor computing a hash shows that a model read or understood the source, relied on it, or was entitled to use it. Trusted evidence of a hash computation can support file access by that computing component; a reported digest alone cannot.

An unpredictable quoted-span challenge can supply sampled evidence that the responder can return content at the specified offset under a declared threat model. A guessed, memorized, cached or relayed answer must be accounted for; a successful challenge does not prove historical reading, understanding or causal use. This is one possible content-evidence mechanism, not a universal requirement for every verifier. Its strength also depends on the verifier's independence. The manuscript's rule that "independent verification requires a verifier outside the process capable of forging the receipt" is the standard trust-boundary argument: if the same process chooses the challenge, holds the key, and checks the answer, it can produce passing receipts for content it never handled. Section 8.5 applies the same reasoning to in-process HMACs. This distinction is the basis of the PROV-002 hash-only falsifier and of mutation M03 (valid hash, wrong quoted span).

## 1.7 Scale trees: what "scale" means here

Definition 5 defines a ladder `s_0 > … > s_M` of partitions `P_k`, each refining the previous one: every child chunk has a unique parent containing it. Partition depth `k` increases toward finer chunks. The composition proof uses a separate lift index `j`, with `Q_j` corresponding to `P_{M-j}`; increasing `j` moves from fine base obligations to coarse parents. Three choices in this definition carry the weight of later results.

**Finiteness.** `M` is finite and declared by the implementation. Every localization result in §6 depends on finite enumeration; nothing is claimed about limits.

**Refinement as containment.** Requiring `C ⊆ parent(C)` gives each fine item exactly one coarse ancestor. That is what makes the sticky case (deep dive 03) well defined and what makes failure localization a tree walk.

**Scales are audit levels, not model scales.** "Leaves represent the finest declared units, not unknowable token-level neural semantics." A scale may be a routing layer, a retrieval-to-claim stage, a tool boundary, a handoff, or a chunk refinement. The induction in §6.2 is over these declared levels. It is not induction over parameter count, training compute, or context length, and no result about those is implied. Keeping this explicit prevents the most tempting misreading of "induction on scales."

The manuscript also notes that "no formula for `M` is imported from geometry." In Guth–Wang–Zahl (2026) the scale chain `ρ_k = δ^{k/M}` is part of a proof; here depth is an engineering budget.

## 1.8 Density observables and boolean stickiness

Definitions 6–7 introduce `KT_k` (maximum ratio of selected chunks in a window to the data owner's expected count) and `F_k(alpha)` (maximum supported-mass share in a declared family of short contiguous subspans). For each positive-mass chunk `C`, eligible subspans satisfy `0 < length(V) ≤ alpha_k length(C)`, with `0 < alpha_k < 1`. The family, length convention and measure are declared inputs. Excluding the full chunk repairs the original unrestricted maximum, which was identically 1 because `V=C` was eligible. These computational analogues are not Katz–Tao or Frostman quantities. They can distinguish selected distributions under their chosen windows, but useful thresholds and any relationship to omitted evidence require independent evaluation.

`STICKY_k` requires fresh nesting and governing acknowledgments, with clustering caps added only when the contract declares them applicable: `NEST_OK_k ∧ ANCEST_OK_k ∧ optional([KT_k ≤ θ_KT] ∧ [F_k(alpha) ≤ θ_F])`. Explicit inapplicability is required to omit the caps. If a required observable's denominator, measure, expected count or eligible window family is unavailable, including when no eligible windows or no positive-mass chunks exist, its value is `UNKNOWN`. Missing required contract parameters or thresholds yield `NO_VERDICT`. A cap comparison does not replace nesting or authority checks.

## 1.9 Events, happened-before, and slices

Definition 13's event tuple collects routing, proposals, commits, text, continuity state, reads, writes, actor, authority evidence, lineage edges, and clocks into one record per step, so all four predicate families read from a common trace. Definition 14 defines happened-before as the smallest relation containing program order, send/receive, intake/receipt, proposal/commit, and correction/dependency edges — Lamport's (1978) relation specialized to the event kinds that matter for certification. Wall clocks alone are insufficient across distributed components because clock skew can invert causal order; explicit edges cannot.

Definition 16's slice completeness rule — a predicate whose required inputs are absent returns `UNKNOWN` — is what prevents a truncated log from being certified. It is the foundation of every non-vacuity argument in the paper.

## 1.10 Summary of what this layer establishes

The foundation layer makes three things true by construction: every object a predicate inspects is unambiguous (typed, epoched, versioned); every relationship a predicate relies on is an explicit, witnessed edge in a finite graph; and every absent input produces a named non-pass rather than a default. Those three properties are what allow the later sections to state results about boolean outcomes that are provable rather than aspirational.

---

# 02 — Boolean Evidence Semantics: Why Alignment Can Be Boolean Without Pretending to Know

*Backs manuscript §§1.1, 4.1, 4.6–4.9, 5.1, 6.1, 6.6, Appendix A.1 and B.1–B.2.*

## 2.1 The central claim, stated exactly

The manuscript's thesis is that alignment, in a specific declared sense, "can be treated as a boolean kind of problem." The precise content is a separation of two layers:

1. **Semantic layer.** For an obligation `q`, a fixed history `H_q`, a policy snapshot `Π(q)`, and a specification `Spec_k`, the authorization predicate `A_q` and the compliance predicate `C_q` each take a value in `{0,1}`. Once the specification and history are fixed, either the history satisfies the specification or it does not. This is ordinary bivalence of a well-defined proposition.
2. **Evidence layer.** A monitor observing partial evidence returns a verdict `V ∈ {T, F, U}`, refined operationally into `PASS`, `FAIL`, `UNKNOWN`, and `NO_VERDICT`. The monitor may not know the semantic value.

The release decision is then a third object: a bit `b(P) = 1` iff `Ev(P) = PASS`. That bit is boolean by design. The thesis is not that knowledge is always available; it is that the *question* is boolean and the *control* is boolean, while the *evidence state* is honestly three-valued. Deep confusion about "boolean alignment" usually comes from collapsing those three layers.

## 2.2 Why this monitor retains an inconclusive outcome

Suppose a monitor were required to answer only true or false. Given incomplete evidence about obligation `q`, it must either answer `T` — and be unsound whenever the missing evidence would have shown a violation — or answer `F` — and be unsound whenever the missing evidence would have shown compliance. The manuscript's "Unknown is not true" counterexample (§6.11) makes this concrete: three required operations, two observed and passing. Reporting "2/2 passed" silently changes the denominator. Reporting `F` asserts a violation nobody observed. Within the chosen total-response interface, `U` is the sound response when neither truth value is justified. An abstaining interface or a richer evidence algebra could also preserve soundness; exactly three values are not universally forced.

This is the same structure runtime verification faces. Bauer, Leucker, and Schallhart (2011) define three-valued LTL semantics for finite trace prefixes precisely because a finite observation of an unfinished execution may neither satisfy nor violate a property; an inconclusive outcome preserves soundness in their monitor design. The index-

alignment monitor adopts this design for its own evidence conditions and is in the same position with respect to missing receipts, unavailable assessors, and incomplete causal histories.

## 2.3 Strong Kleene conjunction and Proposition 1

The manuscript aggregates verdicts with strong Kleene conjunction: `F` dominates, `T` is the identity, and otherwise the result is `U`. Proposition 1 proves that if each local verdict is sound (`V_q = T ⇒ P_q = 1`, `V_q = F ⇒ P_q = 0`), the aggregate is sound for the conjunction.

The proof is short, but each case does work:

- **Aggregate `T`.** Strong Kleene yields `T` only if every input is `T`. Soundness gives every `P_q = 1`, so the conjunction is 1.
- **Aggregate `F`.** Some input is `F`, so some `P_q = 0`, so the conjunction is 0 — *regardless of unknowns elsewhere*. This is why an observed failure can be reported even when other evidence is missing; Trace C relies on it.
- **Aggregate `U`.** No input is `F` and some is `U`. The semantic conjunction might be 0 or 1. The monitor correctly asserts neither.

Two consequences matter. First, soundness is *inherited*, not *created*: "it does not make an unsound checker sound." A support model that labels contradictions as support breaks the guarantee at its source, which is why deep dive 08 insists on independent labels. Second, strong Kleene conjunction retains a decisive observed `F` even alongside `U`. Weak Kleene conjunction can remain sound but is less informative in that case because it returns `U`. A different rule that relabels unknown evidence as observed falsehood can be unsound: an aggregate `F` would no longer imply an actual violation. Mapping unknown to a release bit of zero is instead an allowed control choice when the evidence reason remains visible.

## 2.4 PASS, FAIL, UNKNOWN, NO_VERDICT: four reasons, one release rule

At the diagnostic layer the manuscript keeps four statuses:

| Status | Meaning | Required witness (Appendix A.1) |
|---|---|---|
| PASS | required evidence present, fresh, satisfies contract | positive evidence |
| FAIL | present evidence violates contract | identified counterexample |
| UNKNOWN | evidence missing, incomplete, stale, or assessor unavailable | insufficiency reason |
| NO_VERDICT | thresholds or assessor contracts not supplied, or reconciliation exhausted | name of missing contract |

`UNKNOWN` and `NO_VERDICT` both map to `U` semantically, but they demand different repairs. `UNKNOWN` asks the operator to collect evidence; `NO_VERDICT` asks the contract owner to decide something — a threshold, a coverage floor, a round cap. Merging them would

send work to the wrong person. Trace I (missing `min_coverage`) exists to show that a contract gap must never be repaired by quietly assuming a default: if a system picks `min_coverage = 0.9` on its own, it has made a policy decision it had no authority to make.

The reference pseudocode in Appendix B.1 orders the checks deliberately: undeclared obligation → missing thresholds → execution failure → counterexample → positive witness → otherwise `UNKNOWN`. The final fall-through is the important line. Absence of a counterexample does not produce `PASS`; only a positive witness does.

## 2.5 Non-compensation

Coleman (2026a, Definition 2) defined the alignment deficit `Δ_t = λ_1 d_route + λ_2 d_led + λ_3 d_gen + λ_4 (1 − Cont)`. Weighted scores can summarize defects and support optimization; differentiability requires differentiable constituent functions on an appropriate domain. Whether release compensates for a failed required condition depends on the score ranges, weights and release threshold, not on scalar representation alone. For example, if `b_i∈{0,1}`, `w_i>0` and `D=Σ_i w_i(1-b_i)`, releasing exactly when `D=0` is equivalent to conjunction. No term can compensate for a positive deficit because every term is nonnegative.

The manuscript requires a noncompensatory release rule, expressed as conjunction; an equivalent scalar encoding is permitted. §4.9 states the formal property: replacing any required `True` by `False` makes `A(k)` false regardless of other values. A boolean rule that is true exactly when every required input is true is, by definition, conjunction; any rule that can return true while some required input is false is compensatory. The same discipline appears in Coleman (2026b) for SAVER and SIUS grain vectors ("no weighted average compensates a failed grain"), and the manuscript adopts it as the operational meaning of "boolean alignment": named necessary conditions.

Non-compensation is a design choice with costs. A conjunctive release rule may block outputs that a compensatory weighted rule would pass; an equivalent zero-deficit scalar rule makes the same decisions as conjunction. The partial-release machinery (deep dive 06) exists to recover utility without reintroducing compensation: rather than averaging a failure away, it isolates the failure and releases what does not depend on it.

## 2.6 Composite definitions: A(k), ALIGNED_k, ALIGNED(T)

The manuscript defines three related conjunctions and keeps them distinct:

- `A(k) := A_route(k) ∧ A_ledger(k) ∧ A_gen(k) ∧ A_cont(k)` — full per-scale alignment.
- `ALIGNED_k := COV_k ∧ ORDER_k ∧ CORR_k (∧ STICKY_k)` — a narrower coverage/order/correction diagnostic contributed by the Kakeyalogic seat packet.
- `ALIGNED(T) := ∀k, A(k)` — global alignment across scales.

`ALIGNED_k` is not an alias of `A(k)`. A trace can have complete coverage, valid order, and closed corrections and still fail `Fidelity` (Trace C). Naming the narrow diagnostic separately prevents a coverage dashboard from being read as an alignment certificate. Likewise `ALIGNED(T)` is not `CERTIFIED`: certification adds release authority and non-vacuity.

## 2.7 Non-vacuity and the empty denominator

Any universally quantified predicate over an empty set is true. A checker that finds no required routes, no claims, and no obligations will therefore "pass" vacuously. Definition 27 lists the vacuous cases — empty required set, unknown denominator, empty route or claim sets despite required work, skipped checks reported as passes — and Proposition 6 shows that if `NONVACUOUS` is a required conjunct, a non-PASS value forces the release bit to zero.

The companion rule `EvidenceCoverage = |Q_sound|/|Q_req|`, with `n/a` when `Q_req = ∅`, closes the reporting loophole. Reporting 100% coverage on an empty universe is the denominator version of the same error. The manuscript's insistence that the obligation universe be *independently* established (§5.5) — by a specification manifest rather than by counting observed records — follows: the denominator cannot come from the same evidence it is meant to audit. The 10-agent-team notes flagged exactly this circularity in an early version of Trace A ("all five obligations present" includes itself); the manuscript now states that the fifth obligation's completeness comes from the manifest.

## 2.8 SKIPPED is not PASS

§4.7 adds a fifth operational marker: when an earlier failure makes a later check unsafe to run, the later check is `SKIPPED`. This prevents a common reporting artifact in which tests that never ran appear green. It also preserves the rest of the evidence: a certificate reports the first failure *and* all later observed failures, so an early failure does not hide independent later defects.

## 2.9 Boolean predicates are only as good as their specifications

The boolean framing makes one limitation sharper rather than softer. A semantic predicate is boolean only relative to a fixed specification. If the specification is wrong, a correct certificate certifies the wrong thing. §12.6 lists "semantic booleanization error" — forcing a graded or contested judgment into PASS/FAIL — as a failure mode, with the defense of retaining `CONTESTED` and `UNKNOWN` labels and booleanizing only the release control. Deep dive 09 connects this to Grill (2024) on the politics of testing infrastructure: who writes the specification is a governance question the certificate cannot answer.

## 2.10 What this layer establishes

Booleanity in this paper is achieved by three moves: fix the specification so the semantic question has a truth value; report monitor knowledge in three values so that soundness is preserved under incomplete evidence; and project to a PASS-only bit at the control boundary so that release is conservative without mislabeling unknowns as

failures. Proposition 1 shows these moves compose soundly; Proposition 6 shows non-vacuity can be enforced by the same conjunction. The rest of the paper builds on this algebra.

---

# 03 — Scale Refinement and Composition: Induction on Declared Scales

*Backs manuscript §§1.2, 2.2, 5.1–5.9, 6.2, 6.8–6.9, 6.11, Traces A and K.*

## 3.1 What "induction on scales" borrows and what it does not

In three-dimensional Kakeya geometry, Wang and Zahl (2025) prove that Kakeya sets in $\mathbb{R}^3$ have full Minkowski and Hausdorff dimension, and Guth, Wang, and Zahl (2026) give a streamlined proof. Coleman (2026b, §2.4) reads their architecture as an order of work: control the sticky configuration, then the non-sticky sparse two-scale case, then factor via slabs and planks, then account for the residual, and only then state the general result. That order is the part this paper borrows.

What it does not borrow is any inequality. The geometric arguments establish asymptotic volume and dimension bounds stated with $\lesssim$-type inequalities and losses that are acceptable in the limit. An audit needs a threshold comparison at a finite scale. §1.2 of the manuscript says the analogy "changes how we organize an audit; it does not supply the audit's semantic predicates or prove their validity." The Kakeyalogic seat packet makes the same point: boolean, traceable induction on scales is not a consequence of Wang–Zahl or Guth–Wang–Zahl. Everything proved in this deep dive is proved from finite combinatorics and the stated hypotheses.

## 3.2 The obligation universe

At each lift level `j` the manuscript requires an independently specified finite obligation universe `Q_j` (§5.1), corresponding to partition `P_{M-j}`. Partition depth `k` increases toward finer chunks; lift index `j` increases toward coarser parents. An obligation can be local — one activation, one retrieval, one commit — or a *cross-operation invariant* such as a shared budget or a correction frontier. Cross-operation obligations are the entire reason composition is nontrivial; deep dive 03.7 shows what fails without them.

For each obligation `q` the paper uses two predicates, authorization `A_q` and compliance `C_q`, because they answer different questions: "an authorized write can violate a schema; a well-formed retrieved instruction can be unauthorized." Keeping them separate prevents a single combined "ok" bit from hiding which question failed.

## 3.3 The four composition conditions

Proposition 2 lifts sound local certificates to sound global ones under four conditions. The proof uses all four as sufficient premises. They are not claimed to be a minimal axiom set or universally necessary for every alternative architecture. Their failure permits recognizable counterexamples.

**R — Evidence-preserving refinement.** For each parent obligation `q` there is a child set `D_j(q)` with `(∧_{r∈D_j(q)} (A_r ∧ C_r)) ⇒ (A_q ∧ C_q)`. This is a refinement mapping in the spirit of Abadi and Lamport (1991): it states how a coarse specification is implemented by finer steps. The manuscript's R is narrower than their existence theorem; it is a certificate-specific implication that must be witnessed. Renaming an index or matching a label does not supply it.

**C — Complete obligation coverage.** `D_j(q)` accounts for every operation, effect, and cross-operation premise relevant to `q`, and `Q_j` is closed under work generated during the trace — retries, callbacks, dynamically spawned tasks. Without C, the induction proves the parent from an incomplete set of premises.

**N — Noninterference.** Operations outside the certified set cannot alter state on which authorization, compliance, evidence, or refinement depends. This is the noninterference idea of Goguen and Meseguer (1982) applied to certificate premises: low-integrity content must not influence high-integrity decisions. Concretely, retrieved text cannot rewrite policy, routers cannot commit, generators cannot self-accept.

**K — Correction-aware concurrency and freshness.** The contract states operation/transaction boundaries and its required real-time freshness guarantee. Single-object operations may use linearizability, but individually linearizable field reads can straddle an update. Cross-field reads require a joint atomic snapshot, a justified strictly serializable transaction, or validated version protocol across all relevant fields, with appropriate effect-boundary validation. Ordinary serializability is not equivalent to either guarantee (Herlihy and Wing, 1990, §3.3). Relevant reads use the specified snapshot semantics; every causally preceding correction is included; versions are valid at effect boundaries; accepted corrections invalidate dependents before promotion; concurrent conflicts are resolved; replayed approvals do not create fresh authority.

## 3.4 The proof of Proposition 2, annotated

The manuscript's proof is induction over the finite lift index `j`, from `Q_0` at partition `P_M` toward `Q_M` at `P_0`. The annotated version below shows exactly where each hypothesis is consumed.

- **Base.** Every `q ∈ Q_0` has sound `T` verdicts. Soundness converts them into `A_q = C_q = 1`.
- **Step.** Assume all obligations at lift level `j−1` are true with certificates. Take `q ∈ Q_j`.
  - *C* ensures `D_j(q)` contains every relevant premise, so nothing outside the child set needs to be true for `q`.
  - *N* ensures nothing outside the certified set altered those premises after they were certified, and that composition conferred no authority the children lacked.
  - *K* ensures the child certificates refer to current, correction-closed snapshots, so they are still about the state the parent depends on.
  - The induction hypothesis makes every child predicate true.
  - *R* converts true children into a true parent.

- The parent's certificate is the child certificates plus the refinement witness, so every global `T` has a finite path to base evidence.

Removing any hypothesis breaks a specific line. Without C, a missing child can be false. Without N, an uncertified operation can falsify a child after certification. Without K, a child certificate can refer to a superseded state. Without R, true children need not imply anything about the parent. This is why the paper describes the hypotheses as "substantial": they are exactly the places where real systems leak.

## 3.5 Proposition 9: certificates are defeasible

Because the parent certificate is built from child paths and hypotheses, it inherits their defeasibility. Proposition 9 states that if any certificate on the path is invalidated, or R, C, N, or K stops holding, the old parent `T` is no longer justified by Proposition 2. The proposition is careful: it does not say the parent becomes false, only that the old proof no longer supports it. This is the logical basis for treating policy updates, source version changes, and verifier upgrades as part of certificate identity (deep dive 05).

## 3.6 Sticky, non-sticky, and the band

The **sticky case** (§5.3) is the regime in which each fine item has one visible coarse parent, authority and correction scope are shared, all versions agree with the snapshot, and no unlisted channel affects the parent. In that regime the parent check depends only on its declared children and the certificate is small: parent identity, children, receipts, correction acknowledgments, refinement witness.

The manuscript defines a concrete stable-custody nesting from the Riemann-probe packet. For every applicable ancestor `a` of grain `g`, `ANCEST_OK_k` requires an acknowledgment `(a.id, c_id, c_ver)` naming the currently governing correction in `a`'s own registry. `NEST_OK_k` requires fresh parent edges and content hashes. The paper explicitly declines a global comparison `correction_version(parent) ≤ correction_version(child)` because counters from different registries are incomparable — a subtle point: version numbers are only ordered within the registry that issues them.

The **non-sticky band** is the set of grains, claims, and transitions for which nesting fails. The geometric reading in Coleman (2026b) is that coarse tubes can overlap heavily while fine tubes inside them are sparse; the systems reading is that ancestry, authority, or correction scope diverge across children. The discipline imported is *factor only the band*: repair the region that left the sticky regime rather than restarting the whole trace or averaging its failure into global telemetry. Deep dive 05 gives the affected-closure algorithm that does this.

## 3.7 The shared-budget counterexample

§6.11 and Trace K give the canonical demonstration that local passes do not compose. A parent permits total spend 100. Two children each spend 60 and each is individually authorized up to 60. Locally, both `(A, C) = (T, T)`. Globally, `60 + 60 = 120 > 100`, so parent

compliance is false.

The example identifies precisely which hypothesis failed. R and C omitted a cross-operation obligation — the shared cap. No number of additional local checks repairs this, because the violation is a property of the pair, not of either element. The repair is structural: add the shared-cap invariant to `D(parent)`, partition the budget in advance, or use an atomic reservation in the ledger so that the second reservation observes remaining capacity 40 and is rejected. The manuscript records detection and rejection separately, anticipating the selection/gate/execution distinction of deep dive 06.

This counterexample generalizes. Any global property that is not the conjunction of local properties — capacity, uniqueness, mutual exclusion, global ordering, aggregate privacy budgets — requires an explicit cross-operation obligation. The obligation universe must be designed with that in mind.

## 3.8 Retention on a finite DAG (Proposition 8)

AIce's contribution adds a complementary result. For each node `v` of a finite refinement DAG, let `Required(v)` be the governing evidence and transitions needed for its action and `C(v)` the admitted carried set. Under base coverage, edge preservation, admission of newly governing patches before action, valid scoped fresh supersession, an enforced gate on every execution, and no unmodeled execution channel, topological induction gives `Required(v) ⊆ C(v)`.

The result is about retention, not semantic sufficiency: it shows required constraints are carried, not that they are the right constraints. AIce's sources also show why the enforced-gate hypothesis is load-bearing. The shipped simulator was advisory — it could execute a selection the gate had rejected — and a local enforcing wrapper that blocked correctly also produced a false stop on legitimate work (E2). Custody and enforcement are different properties, and both need evidence.

## 3.9 The capped schedule and why a loop is not a proof

§5.9 gives the general schedule: for each scale, establish `Q_j` and versions; if sticky, verify locally with refinement witnesses; otherwise factor the band and repair the affected closure under R/C/N/K; recheck; record residuals; return `STRUCTURAL_READY` only if every level passes. The manuscript is careful that "a finite loop is not by itself an induction proof." The loop is how the proof's premises are gathered; Proposition 2 is what turns gathered premises into a conclusion. `max_rounds` bounds iterations. Total termination additionally requires terminating calls or enforceable per-call timeouts/cancellation, and neither the cap nor a timeout certifies correctness. Residuals — items no admissible carrier explains, the analogue of Grain Zero in Coleman (2026b) — remain visible in the certificate rather than disappearing.

## 3.10 What this layer establishes

Composition across scales is proved valid under four explicit conditions that correspond to well-understood formal traditions: refinement mapping, closed-world coverage, noninterference, and linearizable freshness. The counterexamples motivate preserving these premises; they do not establish minimality or empirical necessity in every system. The sticky/non-sticky distinction proposes targeted repair and visible residuals. Any efficiency or utility advantage remains to be measured.

---

# 04 — Routing, Ledger, and Generative Continuity: The Four Predicate Families

*Backs manuscript §§2.1, 2.3–2.5, 3.3–3.5, 4.2–4.5, 7.1–7.5, Appendix E, Traces B, C, N.*

## 4.1 From four consistency relations to four predicate families

Coleman (2026a, Definition 1) declared a system state `(R_t, L_t, y_{1:t}, C_t)` index-aligned when four relations hold: routing–capability, routing–ledger, generation–index, and continuity–ledger consistency. The continuation keeps that decomposition and replaces each informal relation with a conjunction of named, separately checkable clauses. The crosswalk in Appendix E records the correspondence; this deep dive explains why each clause is in its family and what failure it isolates.

## 4.2 `A_route`: routing is a proposal, not a judgment

```
A_route(k) := ∀r ∈ RequiredRoutes(k), Typed ∧ Live ∧ Permitted ∧ Budget ∧ Parent ∧ Admissible.
```

- **Typed** and **Live** rely on deep dive 01: the index is well-formed and the registry entry it names exists, is valid at this epoch, and matches its digest.
- **Permitted** applies policy `Π_k` for the authenticated principal.
- **Budget** checks declared sparsity, cost, and latency. In MoE terms this covers top-k limits and expert capacity factors (Lepikhin et al., 2020; Fedus et al., 2022); in agent systems, the number of agents activated per step (Coleman 2026a, Definition 4, sparse Mixture-of-Agents).
- **Parent** checks ancestry: a fine route must descend from a declared coarse route.
- **Admissible** applies a task-specific rule when one is part of the contract and must be explicitly declared out of scope otherwise.

The family deliberately excludes semantic expert quality. Routing literature studies load balance, entropy, and router collapse (Cai et al., 2024 survey; Yue et al., 2024 on dynamic expert counts; Cai et al., 2024 Read-ME on router decoupling). Those are optimization concerns. A low-entropy route can be right for a narrow task and a balanced route wrong. §2.3 therefore states that `A_route` "does not declare an oracle expert from routing entropy alone." What it does certify — that the selected index is real, current, permitted, budgeted, and parented — is narrower and checkable.

Trace B shows the family's diagnostic value. Expert `e7` is typed, live, within budget, with normal gates and load balance. Policy permits privileged repository transformations only on `e8`. `Permitted = F`, so `A_route = F` and `FIRST_FAIL = 0 (A_route.Permitted)`. A scalar routing score would have been excellent.

### Soft mixtures

§7.1 notes that a soft mixture with no hard expert identity may fall outside the routing contract. The correct response is reduced scope or `UNKNOWN`, not an invented discrete index. This keeps the framework honest about what is observable: if a system does not expose discrete selections, the routing family cannot certify them.

## 4.3 `A_ledger`: authority lives at the commit boundary

```
A_ledger(k) := PROV ∧ AUTH ∧ TRANSITION ∧ NOSILENT ∧ COV ∧ ORDER ∧ CORR.
```

Coleman (2026a, §3.2) made the ledger the authority for durable commitments: routers and generators propose, only accepted transitions change committed state. That mirrors transactional designs for agent memory such as TARL (Xiao et al., 2026), from which Coleman (2026a) draws its ACCEPT/PEND/REJECT/SUPERSEDE action types. The package's StateLedger citation (Yang et al., 2026) remains unresolved in the independent review and is not relied on as verified support. Magentic-One (Fourney et al., 2024) uses task and progress ledgers in its orchestrator. The continuation's contribution is to name the clauses a ledger must satisfy for its authority to be certified rather than assumed.

The seven clauses are separated in pairs that are commonly conflated:

- **PROV vs COV.** One cited lineage on an accepted record (PROV) does not show that every required unit was received (COV).
- **ORDER vs AUTH.** Happened-before (ORDER) shows which write came first; it does not authenticate the writer (AUTH). Lamport clocks order events; they do not grant permission.
- **COV vs CORR.** Complete intake (COV) does not show that later corrections reached dependents (CORR).
- **TRANSITION vs NOSILENT.** A transition can satisfy schema and still overwrite an accepted fact; NOSILENT requires a supersession link and that the prior record remain queryable.

`AUTH` uses the three-factor authority evidence of Definition 9, `Act(x) = 1` iff `S(x) = H(x) = U(x) = 1`, from the $L^2$_C authority-detection rule in Coleman (2026b, §7.2): authenticated source authority, hierarchy validity, and authorized user intent. The continuation adds a third value: any `0` makes `Act = 0`; otherwise any `UNKNOWN` makes `Act = UNKNOWN`. This is the evidence algebra of deep dive 02 applied to authority, and it is why "coverage, stickiness, a found receipt, causal order, or model agreement never sets `S(x) = 1`."

The name "ledger" confers none of these properties (§7.2). An append-only store can faithfully preserve an unauthorized write; immutability is integrity of history, not legitimacy of content. Trace L — a retrieved instruction that rewrites the authority table — fails N and AUTH while every append-only guarantee holds.

## 4.4 A_gen: citation membership is necessary and insufficient

A_gen(k) := ∀q, Lineage ∧ ActiveEvidence ∧ Fidelity ∧ StatusType ∧ Scope ∧ AttributionCompleteness.

Coleman (2026a) stated generation–index consistency as "claims cite indices that were actually activated." That is the pointer discipline of Vinyals et al. (2015) applied to citations, and it prevents one failure: citing an inactive identifier. It does not prevent the more serious failure of attaching a real identifier to a sentence the source does not support. The attribution literature measures exactly that gap. AIS (Rashkin et al., 2023) defines attribution as whether a generated statement is supported by an identified source and builds human annotation protocols for it. ALCE (Gao et al., 2023) evaluates citation recall and precision separately from fluency and correctness. Wallat et al. (2024) show that citation correctness and causal faithfulness diverge: a cited document can support an answer without being the document the model relied on.

The continuation responds by splitting the relation:

- **Lineage** — every cited index has a witnessed path, no failed cast.
- **ActiveEvidence** — the evidence was visible in a snapshot accessible to the claim.
- **Fidelity** — a declared assessor labels the claim and all required ancestors SUPPORTED; none is UNSUPPORTED or unquarantined CONTESTED.
- **StatusType** — the claim does not assert a stronger kind than its source (strategy reported as proved, analogy as measurement).
- **Scope** — measured claims stay within the tested workflow and validity interval.
- **AttributionCompleteness** — every claim class requiring attribution has a supporting citation.

Trace C is the canonical demonstration: every chunk receipted, order valid, every citation active, and the conclusion says a dose was doubled when the source says halved. Fidelity = F, A_gen = F. Even with Fidelity, the paper keeps one limit explicit: ActiveEvidence certifies the exposed trace, not internal causal reliance — the gap Wallat et al. identify cannot be closed by adding citation IDs.

StatusType and Scope are unusual in generation checks and deserve emphasis. Many harmful outputs are not false; they are overstated. A strategy reported as a proof, or a result measured on one workflow reported as general, passes a support check on its content and fails on its kind. Falsifiers SEM-003 and SCOPE overclaim (§11.9) target these.

## 4.5 A_cont: continuity is preserved obligations, not fluent prose

A_cont(k) conjoins Projection, Obligation, Handoff, CorrectionContinuity, Directional, and NoAuthorityInflation.

Coleman (2026a, §3.3) measured continuity with a score Cont(y, C) ∈ [0,1]. The continuation turns that score into finite obligations that must survive a specific transformation — compression, handoff, resume. The continuity state C_k = (facts,

`discourse, goals, corrections)` must be a declared projection of the committed ledger view; every open obligation must remain open or carry an authorized satisfaction or waiver; every route change, handoff, resume, and compression must emit a checkpoint; active corrections must be carried; required directions of relevance (deep dive 07) must remain covered; and the continuity expert must not have acquired commit authority.

The motivating literature is agent memory. MemGPT (Packer et al., 2023) pages context in and out of a bounded window; Generative Agents (Park et al., 2023) summarize memory streams into reflections. Both compress, and compression is where obligations get lost. Trace N is the canonical case: after compression, the ledger still holds an open obligation to verify a source version, but the summary omits it. Routing, ledger, and generation pass; `Obligation = F`, `Handoff = F`. "A beautifully written continuation can fail Projection; an awkward one can pass."

NoAuthorityInflation enforces Coleman (2026a)'s capability isolation table (Definition 3): the continuity expert may emit text and propose summaries, but owns no factual authority and may not mutate the ledger. A continuity summary can be internally consistent and wrong, and "projection consistency is not factual support, and support is not user authority."

## 4.6 Role-to-predicate matrix

§7.5 assigns each component the predicates that test it and the inference it may not make. The matrix is a compact statement of capability isolation as testable obligations: the router may not infer authority, the retriever may not infer support, the tool gateway may not infer success from selection, the ledger may not infer truth from append order, the generator may not infer support from IDs, the continuity expert may not infer commit authority, and the release controller may not infer promotion from coverage. Each row names a typical first failure, which gives evaluators a concrete starting point for fault injection.

## 4.7 What this layer establishes

The four families operationalize Coleman (2026a)'s four consistency relations as conjunctions of independently failing clauses. The clause boundaries are drawn exactly where the cited literature reports conflation in practice: gate weight vs permission, order vs authority, citation vs support, summary vs commitment. Each family can fail while the others pass, and the worked traces exhibit each such failure.

---

# 05 — Correction, Causal Order, and Freshness

*Backs manuscript §§3.6, 5.7–5.8, 6.7, 6.9, 6.11, 8.1–8.2, Appendix B.4–B.5, Traces E, F, G, H.*

## 5.1 Why corrections are the hard case

A system that never changes its mind is easy to certify. Real systems receive corrections: a source is revised, a policy revoked, a user retracts an instruction, a tool is deprecated. Each correction makes some earlier certificates obsolete, and the difficulty is knowing which. Too narrow an invalidation lets stale conclusions survive; too broad an invalidation discards everything and destroys utility. The manuscript's answer is to treat a correction as a scoped graph operation with explicit authority, and to compute its consequences by reachability.

## 5.2 The correction object

Definition 15: `c = (target, issuer, authority_evidence, scope, applicability, lineage, nonce, tag, timestamp)`. Each field has one job.

- **target** is the seed in the dependency graph.
- **issuer** and **authority_evidence** determine `Authorized(c)`, which requires authenticated authority over the target and scope.
- **scope** and **applicability** prevent a correction to one tenant, workflow, or interval from silently rewriting another (mutation M15).
- **nonce** and **tag** support integrity and replay detection.

The manuscript states that "cryptography does not make the issuer correct." A validly signed correction from an authorized issuer can still be wrong; the protocol ensures only that it is attributable and not replayed. And `Authorized(c)` "is independent of whether many agents repeat it" (§8.1): repetition is not authority. That rule is a direct response to conformity findings in multi-agent settings (Weng, Chen, and Wang, 2025) — models can converge on a group answer whether or not it is correct.

## 5.3 Happened-before rather than timestamps

Lamport (1978) defined happened-before as the smallest relation containing program order and message send/receive, closed under transitivity, and showed that physical clocks in a distributed system cannot by themselves recover it. The manuscript's Definition 14 specializes the relation to the edges that matter for certification — intake/receipt, proposal/commit, correction/dependency — and requires its transitive closure to be acyclic.

Three outcomes are distinguished. A cycle in complete relevant evidence is `ORDER = False`: the trace claims something impossible. Missing required edges are `UNKNOWN`: the order cannot be determined. An absent ordering contract is `NO_VERDICT`. This three-way split matters because the repair differs: a cycle indicates corrupted or forged records, missing edges indicate incomplete instrumentation, and an absent contract indicates a design gap.

## 5.4 The stale-concurrent-approval example

§6.11 gives the example that motivates condition K. Approval `a` authorizes `tool:t17` under registry `r7`. Correction `c` revokes it in `r8`. Operation `o` depends on `a` but not on `c`. Vector clocks `VC(a) = (1,0,0)`, `VC(c) = (0,1,0)`, `VC(o) = (1,0,1)`.

Read componentwise: `VC(a) ≤ VC(o)` in every coordinate and differs, so `a → o`. `VC(c)` and `VC(o)` are incomparable — `c` has a 1 in the second coordinate where `o` has 0, and `o` has a 1 in the third where `c` has 0 — so `c` and `o` are concurrent. The operation's causal past contains the approval and not the revocation.

The monitor's verdict depends on what it knows about the correction frontier. Before the frontier is known, current authorization is `U`: the approval might still be valid. After a complete history shows that current policy requires `r8` validity and `r8` revokes the tool, it is `F`. In neither case is it `T`, even though the signature on `a` verifies. The lesson the manuscript draws — "signed and fresh are separate" — is the precise distinction between integrity of an artifact and its currency. Trace E develops the same example at the tool-call boundary.

## 5.5 Condition K in six parts

Condition K (§5.7) is the formal summary of what freshness requires:

1. The contract declares operation/transaction boundaries and the required real-time freshness property. Linearizability applies to single-object operations; strict serializability can supply real-time transaction order for multi-operation transactions. Ordinary serializability alone is insufficient for that order and is not equivalent to linearizability (Herlihy and Wing, 1990, §3.3). Individually linearizable field reads do not ensure a joint snapshot: use a joint atomic snapshot, a justified strictly serializable transaction, or a validated version protocol across all relevant fields. Policy-dependent effects require an appropriate atomic admission or revalidation boundary.
2. Every causally preceding correction is included — and concurrent corrections are not fabricated into dependencies. Overstating dependencies is also an error: it can invalidate certificates that were never affected.
3. Versions remain valid at effect boundaries or are revalidated. Time-of-check/time-of-use gaps are the classic violation.
4. Accepted corrections transitively invalidate dependents before promotion.
5. Conflicting concurrent operations are ordered or explicitly resolved.

6. Replayed approvals and nonces do not create fresh authority (mutation M11).

The manuscript also separates four retrieval outcomes that are routinely conflated: `NOT_YET` (causally invisible), `ABSENT` (legitimately nonexistent), `LOST` (failed retrieval), and `FOUND`. "Causal invisibility is not nonexistence, failed retrieval is not a legitimate negative, and retrieval is not permission."

## 5.6 Affected closure

Given seeds `Z` — failing or corrected nodes — the manuscript computes `Aff = Reach_G(Z)`, the set of all nodes reachable from the seeds in the dependency graph, across indices, scales, claims, ledger projections, and pending actions (§5.8). Every node in `Aff` is marked stale or quarantined with the appropriate status, history is preserved, evidence outside `Aff` is retained if N holds and its freshness is valid, repairs are made along affected edges, and the *entire* closure is rechecked — not only the seed.

Rechecking the whole closure is the critical step. A common incremental-repair bug is to fix the seed and assume downstream results update automatically. If a summary was generated from a claim that was generated from a corrected document, fixing the document does not fix the summary; the summary must be regenerated or quarantined. Mutation M14 (one affected descendant omitted) targets exactly this.

Computing `Aff` is linear in the graph: breadth- or depth-first search visits each node and edge once. Appendix B.4 gives the pseudocode, and Proposition 10's bound includes one shared traversal for the union of closures.

## 5.7 Proposition 7: what stays valid

Proposition 7 states the complementary guarantee. If output set `O` satisfies `Deps(O) ∩ Aff = ∅`, N holds, evidence versions for `O` are stable, and the dependency graph is complete for the release scope, then a correction confined to `Z` and its outgoing dependencies does not change the predicates certified for `O`.

The proof is reachability plus isolation: no dependency path from `Z` reaches `O`'s dependencies, N excludes unmodeled influence, and stable versions keep inputs fixed. Each hypothesis has a practical failure. An incomplete dependency graph misses an edge — for example a shared cache or a hidden context window (mutation M31). A failure of N allows a correction to influence `O` through a side channel. Unstable versions mean `O`'s evidence changed for an unrelated reason. When any is uncertain, unaffectedness is `UNKNOWN`, and partial release is not certified.

The manuscript adds a limit that graph semantics cannot overcome: "previously emitted unretractable copies cannot be recalled by graph invalidation." Invalidation governs future promotion. It does not retrieve an email already sent.

## 5.8 Trace G and Trace F

Trace G applies both propositions. A source correction moves `d4` from version 2 to 3. Claim `q4`, summary `s4`, and pending tool action `x4` depend on `d4`; extraction `q1` depends only on `d1`. With dependency enumeration and N independently established, `Aff = {d4, q4, s4, x4}`. Those are held. `q1` may be released as `HELD_PARTIAL` with an explicit exclusion notice, after its own authority and support checks pass.

Trace F applies closure to concurrency. Agent A commits a valid plan; agent B uses a revoked memory item; agent C drops an unrelated open obligation. B and C are causally incomparable. The frontier contains both, repair computes two closures, and A may be released only if its dependencies intersect neither.

## 5.9 Correction storms and the round cap

Trace H is the adversarial case for closure: two authorized registries alternately supersede conflicting values. Each correction correctly invalidates the other's dependents, and the system can oscillate indefinitely. The manuscript's reconciliation loop (Appendix B.5) has a caller-supplied `max_rounds`; exhausting it returns `NO_VERDICT("CORRECTION_STORM_OR_ROUND_CAP")`, or `REFUSED` if policy prohibits unresolved contested output, or `AWAITING_HUMAN` where the contract names an adjudicator. "No arithmetic average of versions chooses a winner." The cap bounds iterations. Total termination of the decision requires every repair/check/I/O call to terminate or a supervisor that can return a deadline outcome independently of a blocked call. A timeout does not establish cancellation or rollback; unconfirmed effects remain held. A timeout is an execution status, not evidence that a registry is wrong; the cap does not decide which registry is right. That decision is an authority question outside the loop.

## 5.10 Freshness as part of certificate identity

The freshness table in §8.2 lists, for each predicate, the evidence that makes a certificate current and the event that invalidates it — a version bump for COV, a rewrite without supersession for ORDER, a new applicable correction for CORR, deregistration or policy revocation for ROUTE_OK, assessor drift for SUPPORT/FID. The table operationalizes Proposition 9: the certificate's identity includes the versions of everything it relied on, so a change in any of them is a change in what the certificate is about. "A preserved artifact can be obsolete; custody is not freshness."

## 5.11 What this layer establishes

Corrections are handled as authorized, scoped seeds whose consequences are the reachable closure in an explicit dependency graph. Happened-before, not timestamps, determines what a step could have seen; declared real-time snapshot/effect semantics constrain "current policy"; one traversal computes invalidation closure on the supplied

graph in linear time; completeness and isolation remain explicit assumptions of partial preservation. The stale-approval and storm examples show both why these conditions are needed and where their guarantees stop.

---

# 06 — Custody, Authority, and Release

*Backs manuscript §§2.2, 3.3 (Definition 9), 5.10, 6.6–6.8, 7.2, 8.3–8.5, 10.4, Traces J, L, M.*

## 6.1 Three things that are easy to confuse

A system can *hold* an object, be *entitled to act* on it, and *release* conclusions drawn from it. These are three separate properties. Coleman (2026b, §6.3) states the first separation as "held = relevant + retained ≠ truth": keeping something for evaluation is not endorsing it. The L²_C authority rule (Coleman 2026b, §7.2) states the second: detection of source, authority, intent, and harm precedes incorporation, and untrusted text may be read as content without becoming an instruction. This deep dive explains how the continuation turns those principles into a release contract.

## 6.2 Custody is not truth

In Coleman (2026b), the HELD predicate requires an object to be typed, grounded, evidence-exposed, correction-preserved, and status-visible. Every clause concerns how the object is kept, none whether it is true. The continuation preserves that property: a grain record (§5.2) carries a content hash, status, authority stamp, dependencies, and governing acknowledgments, and the manuscript states that the hash "is not a read receipt, semantic support, freshness, or permission." A perfectly custodied false claim is still false. Custody is what makes later correction possible — you cannot invalidate what you did not keep — which is why NOSILENT requires superseded records to remain queryable.

## 6.3 Authority: `Act = S · H · U` and `Act = 0 ⇏ Stop`

Definition 9 adopts the three-factor authority gate: authenticated source authority `S`, hierarchy validity `H`, authorized user intent `U`. All three must be 1 for `Act = 1`. Two consequences are built into the definition.

**`S = 0 ⇒ Act = 0`.** Content from a source without authority cannot authorize an action no matter how it is phrased. This is a proposed structural constraint against instruction injection, whose protection depends on complete mediation and enforcement: Trace L's document saying "make router an approver" has a valid content receipt and no authority. The manuscript's "found content is not an instruction" example states the verdict: `F` if evidence establishes no authority, `U` if authority evidence is incomplete, never `T` from retrieval alone.

**`Act = 0 ⇏ Stop`.** Denying an action is not the same as halting the system. A privileged tool call can be refused while an informational response explaining the refusal is still produced. Coleman (2026b) states this as detection without resistance: L²_C "does not resist instruction-shaped content," it declines to incorporate unauthorized instructions

and proceeds toward an authorized outcome. The continuation uses the rule to justify partial release (§6.6 below) and is explicit that it "does not override a separate policy requiring complete suppression."

## 6.4 Selection, gate verdict, execution, effect

AIce's contribution adds a four-way distinction that the manuscript adopts in §§6.8, 7.2, and 10.4: `selected_action`, `gate_verdict`, `executed_action`, `observed_effect`. Each can diverge from the others.

- A gate can reject a selection that an advisory executor runs anyway. AIce's shipped simulator was advisory in exactly this way.
- An enforcing wrapper can block correctly and still reject legitimate work — AIce's E2 false stop.
- A blocked action can still have produced partial logs or externally visible effects before the block.

The manuscript draws the consequence: "a correct verifier can be attached to a broken executor." Certificates about detection say nothing about enforcement unless enforcement is separately recorded and checked. Mutation M27 (gate rejects but tool executes) is classified as `EXECUTION_ERROR`, distinct from detection — a distinction formalized in deep dive 08.

## 6.5 Structural readiness and full release

§5.10 defines two levels of promotion.

`STRUCTURAL_READY_k` is the conjunction of COV, ORDER, CORR, ROUTE_OK, LEDGER_OK, GEN_OK, `A_cont`, and STICKY where required, achieved directly or after capped reconciliation. It certifies that the structure is intact: evidence received, order valid, corrections propagated, lineage complete.

`RELEASE_FULL` additionally requires CROSS_OK, CROSS_SCALE_OK, SCOPE where measured claims occur, POLICY_OK, AUTH, SUPPORT, FID, and NONVACUOUS. These checks use evidence independent of ingestion receipts, and SUPPORT/FID use a declared external assessor.

The split exists because structural integrity is necessary and may be cheaper to check under the restricted cost model, while semantic support can be expensive and depends on an assessor whose reliability must itself be established. Reporting structural readiness as if it were release would recreate the coverage-equals-fidelity error at the level of the whole certificate. The Riemann-probe packet contributed this split and the rule that four-valued evidence maps to PASS-only bits, with missing evidence blocking full promotion but not authorized unaffected partial output.

## 6.6 Five outcomes

`INDEX-ALIGN-CERT` returns one of five outcomes (§8.3):

| Outcome | Condition | Typical trace |
|---|---|---|
| CERTIFIED | all full-release conjuncts pass, independent release authority, non-vacuous | A |
| HELD_PARTIAL | only authorized unaffected content emitted; held dependencies named | G, M |
| REFUSED | observed failure prohibits the requested release | B, C, E |
| AWAITING_HUMAN | contract names an authorized adjudicator for an exact question | H (with adjudicator) |
| NO_VERDICT | missing thresholds, incomplete evidence, unavailable assessor, exhausted reconciliation, invalid execution | I, H (no adjudicator) |

The outcome set is richer than pass/fail because control decisions are richer than truth values. The same predicate trace can yield different outcomes under different authorized scopes: "a semantic pass in a sandbox can coexist with refusal to publish because AUTH is absent." `AWAITING_HUMAN` names a role and an exact question, not a generic request for oversight.

## 6.7 The partial-release rule

For candidate output `o`, partial emission requires `Deps(o) ∩ Aff = ∅`, that `o`'s own route, authority, policy, support, scope, and continuity checks pass, that dependency enumeration and N pass for its scope, and that the output is labeled partial and names excluded obligations (§8.4). Appendix A.1 adds that `HELD_PARTIAL` "cannot be inferred solely from Aff disjointness."

Trace M shows the rule's purpose. A report has independent sections X and Y; Y's support assessor is unavailable. Full promotion is blocked with `SUPPORT(Y) = U`. X, independently supported and isolated from Y, may be emitted as `HELD_PARTIAL`. Y is not labeled false. Without partial release, a conjunctive rule would discard X because of an unrelated unknown — exactly the utility loss that makes conservative certification unattractive in practice. With it, conservatism is confined to the affected region.

The Grok Bot packet contributed the corresponding operator-facing protocol: labeled partial answers that name uncovered chunks, and `NO_VERDICT` where thresholds are missing.

## 6.8 Independent verification

§8.5 applies the trust-boundary argument of deep dive 01 to the release decision. PROV-002 requires a verifier outside the process capable of forging receipts. A challenge generated and checked by the same process is an honest-workflow control, useful against accidents and useless against a compromised process. An in-process HMAC does not protect against code holding the same key. "The release decision itself is verifier output, not evidence of its own correctness" (§10.1). Independence also has a social dimension: model agreement is not independence, because models share training data and rubric blind spots (deep dive 08).

## 6.9 `h < 1` and private facts

Coleman (2026b, §7.3) treats `h < 1` as an evaluator non-sovereignty boundary: the evaluating system remains bounded, and claimed sovereignty overreach is telemetry. The continuation keeps it as a stipulated design boundary, not a measured score, and adds the rule that private personal facts are excluded from synthetic predicates and cannot raise or lower a verdict. Excluding a class of inputs from the decision is a structural privacy property of the certificate; it is not a proof that the surrounding system preserves privacy.

## 6.10 What this layer establishes

Holding, authorizing, and releasing are separated into distinct evidence channels with distinct checks. Authority is a conjunction of three externally evidenced factors that content cannot supply. Detection is separated from enforcement and enforcement from effect. Release is graded into five outcomes so that a blocked full release does not suppress authorized, isolated, labeled partial output. Together these turn Coleman (2026b)'s custody and authority principles into a release contract whose every clause names its own evidence.

---

# 07 — Failure Localization and the Worked Traces

*Backs manuscript §§4.7–4.8, 6.3–6.5, 7.4, 9.1–9.15, 12.2, Definitions 17, 24–26.*

## 7.1 Why localization is the practical payoff

An aggregate alignment score without component records need not identify a failure location. A scalar representation accompanied by the same component records can retain that information. The boolean trace `𝔅(T) = (b_0, …, b_M)`, with sub-conjunct outcomes and witness references, can. Coleman (2026a) motivated the index abstraction partly through debuggability: separating roles makes failures attributable. The continuation makes that attribution a computable function of the trace.

## 7.2 FIRST_FAIL and FIRST_NONPASS

Definition 24 declares a partition-depth order `π=(k_0,…,k_M)` and uses `FIRST_FAIL(T;π) := min{i : A(k_i) = False}`, recording both order position and partition depth. In the finest-first traces, `i=j` and `k_i=M-j`. Proposition 3: on a finite total scale order, if the failed set is nonempty, the minimum exists and is unique; with a declared sub-conjunct order, the reported failing predicate is unique too. The proof is the well-ordering of finite totally ordered sets.

The manuscript reports a second location, `FIRST_NONPASS`, the least scale with an `UNKNOWN` or `NO_VERDICT` outcome preceded only by passes. The two are reported separately because they mean different things. If scale 2 is `UNKNOWN` and scale 4 is `FAIL`, the observed failure is at 4, but the earliest point where certification became impossible is 2, and the real defect may be there. Overwriting either location with the other would lose information.

The manuscript also limits the claim: `FIRST_FAIL` is "the earliest known failed declared lift, not a metaphysical origin." Several defects can produce the same failed predicate, and the true cause may lie outside the instrumented trace. Trace N makes this concrete: the omission surfaces at the handoff scale even if the compressor that caused it is upstream and unrepresented.

## 7.3 Concurrent failures need a frontier, not a minimum

When the scale structure is a DAG with causal order $\prec$ rather than a total order, two failures can be incomparable. Choosing one as "first" would encode an arbitrary scheduling artifact as a diagnosis. Definition 26 defines `Frontier(F) = {x ∈ F : no y ∈ F has y ≺ x}`, the set of minimal failures. Proposition 4: in a finite acyclic order, every nonempty `F` has at least one minimal element, and every failure has a minimal failed ancestor. The proof is descent along strict predecessors: acyclicity forbids repetition,

finiteness forces termination. Return exactly this minimal set. Adding any failure incomparable with merely one frontier member can add a nonminimal descendant: with `a→c` and independent `b`, the frontier is `{a,b}`, not `{a,b,c}`.

Trace F returns `Frontier(F) = {B.A_ledger.CORR, C.A_cont.Obligation}`. A display may list B first, but the certificate preserves their incomparability. If order edges are missing, the result is `UNKNOWN_ORDER`, consistent with the evidence semantics of deep dive 02.

## 7.4 Computability (Proposition 5)

If each checker terminates on its finite slice, reachability is over a finite graph, and thresholds are present, the trace, frontier, and affected closure are computable in finite time. The proposition is deliberately weak: it is a termination guarantee, not a cost bound, and it excludes semantic discovery. Its practical point is the converse — when a checker does not terminate (an LLM assessor times out), that is an execution status, not a semantic failure. Deep dive 08 develops the execution taxonomy.

## 7.5 Two-ends support and directional coverage

Two predicates detect a failure that counting receipts misses: concentration.

`TWO_ENDS(q)` (Definition 25) requires that no admissible window shorter than `ε|S|` contain more than a `θ_2E` share of a claim's declared support. A claim that is "supported by" a 300-page source but draws all its support from one paragraph may be correct, but it has not engaged the source as a whole; where the contract requires distributed evidence, that should be visible. The vocabulary is borrowed from the two-ends condition in the restriction literature (Wang and Wu, 2024), where it excludes configurations whose mass concentrates in a short piece of a tube; here it is a caller-parameterized audit check, not a theorem.

Directional coverage (Definition 17) requires that each declared direction of relevance — evidence class, obligation, tenant partition — retain a distinguishable lineage at each scale. It is the systems analog of the light-rule posture in Coleman (2026b, §3), contributed through the L seat: every required direction through the custody center remains represented. The manuscript bounds it twice: directions must be declared (no claim to capture all semantic directions), and coverage overlap does not imply authority. High coverage in one class cannot repair an omitted class.

## 7.6 The position-effect motivation

Liu et al. (2024) showed that language models' use of long contexts depends on where relevant information sits, with degraded performance when it is in the middle. That result motivates Trace D, in which beginning and end chunks have challenged quotations and the middle has only a file hash. The manuscript's verdict is `COV_middle = U`, not `F`: the certificate reports missing evidence rather than asserting that the model omitted the middle. The position-effect literature motivates stratifying evaluations by location; it does not license assuming every model fails there.

## 7.7 The diagnostic ladder

The Grok Bot packet contributed a ladder `chunk → section → source → cross-source → handoff`, adopted in §7.4 as a preregistered schedule per task. Each rung has its own obligation universe and transfer witness; passing a rung does not substitute for a missing later rung; skipped rungs must be justified as inapplicable. The ladder is the concrete instance of the declared scale tree for document-grounded tasks, and it makes the first failed lift interpretable: a failure at "section" means within-source aggregation failed, a failure at "cross-source" means synthesis across sources failed, and so on.

## 7.8 The fourteen traces as a coverage argument

The worked traces in §9 are hand-constructed illustrations of selected distinctions and intended verdicts. They are not a complete independence argument or an executed coverage test. Their partial map is:

| Separation demonstrated | Trace |
|---|---|
| Clean conditional lift with independent denominator | A |
| Registered ≠ permitted | B |
| Coverage ≠ fidelity | C |
| Missing evidence ≠ observed failure | D |
| Signed ≠ fresh | E |
| Concurrent failures → frontier | F |
| Scoped invalidation and partial release | G |
| Iteration cap ≠ total termination or correctness | H |
| Missing threshold → NO_VERDICT | I |
| Detection ≠ process exit | J |
| Local passes need cross-operation obligations | K |
| Immutable history ≠ isolation | L |
| Partial release under unrelated unknown | M |
| Fluent continuation ≠ preserved commitments | N |

The hand-constructed traces illustrate failures in each predicate family (route in B, ledger in E/F/L, generation in C, continuity in N) and several outcomes (REFUSED in B/C/E, HELD_PARTIAL in G/M, NO_VERDICT in H/I, and a possible AWAITING_HUMAN branch in H when an adjudicator is declared). K illustrates an omitted shared obligation/refinement premise, L outside interference, and E freshness failure. This is a partial coverage map, not an exhaustive pairwise-separation proof. Additional distinctions, including vacuity, status inflation and evaluator independence, require explicit fixtures and executed results. The traces and 32 mutation designs are not test executions.

## 7.9 Trace A as the positive control

A localization scheme is only meaningful if it also certifies clean cases. Trace A fixes policy, registry, authority, snapshot, and `min_coverage = 1.0`; an independent manifest lists exactly five obligations; every finest induction-level-0 predicate passes on explicit evidence; the induction-level-1 lift has R/C/N/K witnesses. `STRUCTURAL_READY = T`, and with the full-release checks the certificate is `CERTIFIED`. Trace I then reuses Trace A's records with `min_coverage` omitted and obtains `NO_VERDICT` — a minimal pair isolating the effect of a missing contract.

## 7.10 What this layer establishes

Localization is exact where order is total (unique first failure), honest where order is partial (a frontier set), and bounded in its claims (earliest known failed lift, not root cause). Declared concentration and direction checks can distinguish support distributions that identical totals conceal; their usefulness depends on the chosen windows and thresholds. The worked traces illustrate selected separations. Exhaustive fixture coverage and empirical detection remain evaluation obligations.

---

# 08 — Evaluation, Falsification, and Cost

*Backs manuscript §§6.10, 10.1–10.4, 11.1–11.10, Appendix D, Trace J.*

## 8.1 What a falsifiable certificate design looks like

A certification scheme is scientifically useful only if it can be shown to be wrong. The manuscript states its falsifiers as fixtures: inputs for which a wrong design passes and the correct design does not. A fixture falsifies a *named design or implementation conformance condition*. That scoping is precise rather than defensive — a fixture about coverage and fidelity tests whether the release rule conflates them; it cannot test a theorem in geometry or number theory, because those are not inputs to the checker.

## 8.2 The eight core fixtures

From the Riemann-probe packet, §11.9 lists eight fixtures, each targeting one separation:

1. **COV-pass/FID-fail** — all chunks receipted, conclusion unsupported. A design that releases on coverage alone passes it.
2. **Route substitution** — registered but unauthorized tool or expert. A design that ignores `Π` passes.
3. **Stale correction handoff** — child uses pre-correction view. A design that does not invalidate `Aff` passes.
4. **SEM-003 status inflation** — strategy labeled proved. A design without `StatusType` passes.
5. **SCOPE overclaim** — measured claim generalized. A design without tested-workflow scope passes.
6. **PROV-002 hash-only reread** — hash presented as reading. A design without independent content-bound verification passes.
7. **ACT conflation** — covered but unauthorized instruction. A design in which coverage grants `Act` passes.
8. **Index echo** — cited identifier absent from visible activations. A design without generation–index lineage passes.

Appendix D extends these to thirty-two mutations (M01–M32) spanning registry rollback, wrong quoted spans, post-rationalized citations, omitted callbacks, replayed nonces, cross-registry acknowledgments, authority pollution, router self-commit, generator self-certification, empty universes, missing thresholds and assessors, compression-dropped goals, gate bypass, harness crashes, correction storms, hidden caches, and silent rubric edits. Each row states the clean expectation, the fault expectation, and what non-detection would warn about.

## 8.3 Detection requires a baseline and an attributable failure

The most consequential methodological rule in the evaluation section is that "a nonzero exit status alone never counts as detection" (§11.6). It comes from a concrete hazard reported by the 10-agent team's review of an open-addressing probe: same-author mutation scripts that count any nonzero exit as a "killed" mutant. An import error, an unrelated assertion, or a timeout then counts as successful detection, inflating the kill rate with infrastructure failures.

The manuscript's taxonomy separates five outcomes:

- **CLEAN_PASS** — the unmodified baseline passes named checks under the same configuration. Without it, no comparison is valid.
- **DETECTED** — the variant runs far enough to evaluate the intended property and a predeclared named check fails for the intended reason.
- **UNDETECTED** — a valid run completes without relevant detection.
- **HARNESS_ERROR** — the intended check could not be evaluated: import failure, malformed harness, unrelated exception, timeout.
- **EXECUTION_ERROR** — enforcement or environment fails even though detection is valid.

Trace J demonstrates all of them on one configuration. The team notes used `HARNESS_ERROR` and earlier text used `EXECUTION_ERROR` for overlapping cases; the manuscript harmonizes them by putting them on different axes — evaluation infrastructure versus enforcement — so a single run can be `DETECTED` on the detection axis and `EXECUTION_ERROR` on the execution axis.

## 8.4 Labels must be independent of the monitor

§11.2 requires that ground truth come from independently constructed specifications, hand-audited synthetic traces, or injected faults with known mechanisms: "the monitor's own output cannot supply its labels." This is the evaluation analogue of the independent-verifier rule. Three bodies of work motivate the specific protocol.

**Human annotation discipline.** AIS (Rashkin et al., 2023) and FABLES (Kim et al., 2024) build attribution and faithfulness labels through structured human annotation, and FABLES specifically annotates omissions in long summaries. The manuscript adopts claim-level sampling and omission checks from that discipline.

**Independence of judges.** Sycophancy research (Sharma et al., 2024) shows models adjusting answers toward perceived user preferences, and BenchForm (Weng, Chen, and Wang, 2025) shows conformity to group answers. The protocol therefore collects human ratings independently before discussion and treats agreement between model assessors as non-independent evidence: separate answers from models with shared training data and rubrics are correlated.

**Criteria drift.** EvalGen (Shankar et al., 2024) found that users refine their evaluation criteria while grading outputs. The protocol does not deny drift; it versions it. Exploratory rubric development creates a new specification epoch; confirmatory runs

use a frozen manifest (§7.6, §11.8).

## 8.5 Six research questions

The manuscript frames six questions, each paired with a measurable outcome: sound booleanization (inter-rater and oracle agreement under fixed specs), localization (does first failure predict the repair site better than scalar telemetry?), composition (do R/C/N/K witnesses prevent false global acceptance?), partial release (is useful unaffected output retained without contaminated promotion?), cost and utility (latency, tokens, dollars, human minutes, false blocks, missed coverage), and drift (how often do criteria, registries, sources, and assessors change enough to invalidate certificates?). They are posed as questions; their answers are what the protocol is designed to measure.

## 8.6 Metrics that do not hide unknowns

Detection metrics are standard — TPR, FPR, precision on valid evaluable episodes, with `n/a` for empty denominators — with one modification: `U` is reported separately and is never encoded as detection or non-detection. Where policy holds on `U`, the protocol measures that control choice by its false-block and missed-violation consequences "without changing the epistemic label."

Trace-level metrics include global-violation recall, R/C/N/K failure recall by category, false global acceptance, false-block rate, detection latency, correction propagation completeness, stale-approval acceptance, and lineage completeness. The manuscript names the two degenerate designs these metrics must expose: a certificate that refuses everything has zero false acceptance and no utility; one that accepts everything has high completion and no safety value.

Evidence metrics include `EvidenceCoverage` with a vacuity flag, unknown rate by reason, threshold-supplied rate, content-bound vs hash-only coverage, fresh-snapshot coverage, asynchronous side-effect coverage, refinement-witness coverage, and correction-frontier completeness.

## 8.7 Baselines and ablations

The proposed protocol compares the certificate against scalar weighted deficit (Coleman 2026a's `Δ_t`), raw model self-assessment, citation-membership only, receipt-coverage only, a global end-of-trace judge, per-call local checks without composition conditions, and human review alone. It also requires a scalar encoding of exactly the same noncompensatory policy, such as a positive-weight nonnegative deficit released only at zero. This separates policy effects from representation effects; scalar does not itself mean compensatory. Capability, context, tool access, token budget, and human time are matched. Ablations remove typed lineage, content-bound receipts, noninterference checks, correction closure, cross-scale receipts, independent assessors, non-vacuity, and partial-release logic, one at a time and jointly — joint ablations matter because C interacts with N and K with correction propagation.

RAGChecker (Ru et al., 2024) is the closest existing diagnostic framework: it separates retriever and generator metrics to locate defects. The index-alignment protocol shares that modular aim and differs by making each relation a non-compensatory release conjunct with a witness path.

## 8.8 Cost of verification

Proposition 10 bounds verification of a supplied restricted certificate at `O(n + |E| + cB + r + w)`: one pass over records, one shared graph traversal, at most one support-oracle call of cost `B` per claim, plus receipts and windows. The proof charges each item once; the restrictions are stated so that repeated per-claim or per-seed traversals, if an implementation needs them, are charged separately rather than hidden.

The bound is about verification, and §10.4 separates four costs that are often conflated: search (finding routes, evidence, repairs), verification (checking the witness), enforcement (blocking execution), and observed effect (what happened). Cheap verification does not imply cheap search. The P-NP seat's non-claims D1–D9 make the boundary explicit: no reduction is given in either direction, no hardness or membership claim is made for general index alignment, and polynomial verification of a supplied witness does not classify the search problem.

Cer-Eval (Wang et al., 2025) and Derailer-Rerailer (Wan et al., 2025) motivate selective, staged verification to reduce cost; VeriLA (Sung, Kim, and Zhang, 2025) motivates explicit, human-centered agent failure criteria. The protocol adopts the cost-reporting discipline and requires that any selection policy itself be evaluated for missed violations. The Grok Bot packet's rule that verification spending should be incurred where it can change the answer is the operational form.

## 8.9 Existing construction evidence

§11.10 reports what neighboring artifacts already show. AIce's opto probe at revision `e623a10` reports 110 synthetic runs with zero mismatches against frozen expectations and 539 tests, with documented limitations including handoff RPL = 0, live `L_N = .75` versus compressed 1, order-unaware replay, and in-process registry forgery. The open-addressing probe reports 7 of 8 confirmatory predictions held, with P3 failed. These are source-reported numbers for neighboring constructions; their methodological value — failed predictions kept visible, exploratory sweeps separated from confirmatory ones — carries over directly. The `thinkingmachines.txt` transcript documents Benevolence Drift Case 01, an evasive first response that became an explicit "Yes" after a forced binary prompt; Case 02 has no forced-binary follow-up in the supplied material.

## 8.10 What this layer establishes

The evaluation design specifies how to break the construction: fixtures for each separation, a mutation inventory, independent labels, an execution taxonomy that cannot count crashes as detections, metrics that keep unknowns visible, baselines that

include the scalar deficit the paper replaces, ablations, and a cost model that separates verification from search and enforcement. Each element is traceable to a specific hazard reported in the cited literature or in the team's own probes.

---

# 09 — Related Work and Annotated Bibliography

*Backs manuscript §§2, 12 and the References section. The initial package reported a metadata/URL check in `audit/reference-verification.md`. The subsequent independent review distinguishes source identity from claim support and leaves StateLedger unresolved. A resolving link or matching title is not full-text validation or empirical replication.*

## 9.1 How the literature is used

The manuscript draws on five bodies of work, and uses each for a different purpose. Sparse-computation and agent papers supply the *objects* — the indices the certificate inspects. Attribution and evaluation papers supply the *failure modes* the predicates are built to separate. Formal-methods papers supply the *proof techniques* for composition. Kakeya geometry supplies an *order of work*. Governance work supplies the *boundary* of what a technical certificate can settle. Keeping those roles distinct is itself part of the argument: no cited work is used as evidence for something it does not study.

## 9.2 Sparse experts and routing

**Shazeer et al. (2017), *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*, ICLR; arXiv:1701.06538.** Introduces noisy top-k gating over thousands of experts with a load-balancing loss. Source of the gated routing decision `R_t = TopK(softmax(W_r x))` used in Coleman (2026a). Its gate is a learned affinity, which is why the manuscript treats gate weight as telemetry.

**Lepikhin et al. (2020), *GShard*, arXiv:2006.16668.** Top-2 routing with expert capacity and automatic sharding. Motivates the `Budget` clause: capacity is a declared, checkable constraint.

**Fedus, Zoph, and Shazeer (2022), *Switch Transformers*, JMLR; arXiv:2101.03961.** Top-1 routing and simplified load balancing; `K_t = 1` case.

**Jiang et al. (2024), *Mixtral of Experts*, arXiv:2401.04088.** Top-2 among eight experts per layer; the concrete MoE configuration in Coleman (2026a).

**Cai, W. et al. (2024), *A Survey on Mixture of Experts in Large Language Models*, arXiv:2407.06204.** Survey of gating variants; cited for the breadth of routing designs `A_route` must accommodate.

**Yue et al. (2024), *Ada-K Routing*, arXiv:2410.10456.** Dynamic per-token expert counts; motivates treating `K` as a declared budget rather than a constant.

**Cai, R. et al. (2024), *Read-ME*, arXiv:2410.19123.** Decouples the router from the backbone; cited in Coleman (2026a) as inspiration for router/authority separation.

## 9.3 Pointers, retrieval, and indexing

**Vinyals, Fortunato, and Jaitly (2015), *Pointer Networks*, NeurIPS; arXiv:1506.03134.** Outputs are indices into the input. The generation predicate's `Lineage` clause generalizes this guarantee across stores.

**Lewis et al. (2020), *Retrieval-Augmented Generation*, NeurIPS; arXiv:2005.11401.** Generation conditioned on retrieved passages; the baseline setting for `A_gen`.

**Kim and Diaz (2025), *LTRR: Learning To Rank Retrievers for LLMs*, arXiv:2506.13743.** Routing among retrievers; retriever choice is itself an index.

**Wu, M. et al. (2026), *Finding What Matters: Anchoring Context Knowledge with Evolving Indices for Iterative Retrieval*, arXiv:2601.16462.** The current primary version names the method **KAIR**, not GraphAnchor. Its evolving keyword/relation index uses RDF triples and motivates versioned index identity. The source supports this indexing mechanism, not the present certificate's efficacy.

**Liang et al. (2024), *Empowering Large Language Models to Set up a Knowledge Retrieval Indexer via Self-Learning*, arXiv:2405.16933.** Referred to as PG-RAG. A model-built indexer means the index structure itself is a generated artifact whose provenance must be tracked.

## 9.4 Tools, agents, and memory

**Schick et al. (2023), *Toolformer*, arXiv:2302.04761** and **Yao et al. (2023), *ReAct*, ICLR; arXiv:2210.03629.** Expose tool calls and action–observation steps as discrete events, the raw material of the event trace.

**Wu, Q. et al. (2023), *AutoGen*, arXiv:2308.08155; Hong et al. (2023), *MetaGPT*, arXiv:2308.00352; Fourney et al. (2024), *Magentic-One*, arXiv:2411.04468.** Multi-agent orchestration; Magentic-One's orchestrator maintains task and progress ledgers. These systems produce agent indices and handoffs that `A_route` and `A_cont` inspect.

**Packer et al. (2023), *MemGPT*, arXiv:2310.08560; Park et al. (2023), *Generative Agents*, arXiv:2304.03442.** Bounded-context memory management and memory-stream summarization. Compression is where obligations can be lost, motivating `A_cont.Obligation` and Trace N.

**Xiao et al. (2026), *TARL: Transaction-Aware Reliable Ledgers for Executable Memory Management in Long-Term Agents*, arXiv:2608.03699.** Transactional agent memory; source of the proposal/commit separation in Coleman (2026a)'s ledger.

**Yang, Liu, Wu, and Wang (2026), *StateLedger: Path-Addressed External Memory for Persistent Multi-Agent Systems*, claimed NeurIPS 2026 poster. UNRESOLVED.** The given page was inaccessible and no confirming primary record

was recovered in the independent review. This is not proof of nonexistence; the citation remains unverified and is not used to support a substantive claim until a stable primary record is supplied.

## 9.5 Attribution, faithfulness, and evaluation

**Rashkin et al. (2023), *Measuring Attribution in Natural Language Generation Models*, Computational Linguistics 49(4).** Defines Attributable to Identified Sources (AIS). The conceptual basis for separating `Fidelity` from `Lineage`.

**Gao et al. (2023), *Enabling Large Language Models to Generate Text with Citations* (ALCE), EMNLP; arXiv:2305.14627.** Citation recall/precision measured separately from correctness.

**Wallat, Heuss, de Rijke, and Anand (2024), *Correctness is not Faithfulness in RAG Attributions*, arXiv:2412.18004.** A correct citation need not be the one causally used. Grounds the manuscript's limit on `ActiveEvidence`.

**Liu et al. (2024), *Lost in the Middle*, TACL.** Position-dependent use of long contexts; motivates location-stratified coverage tests (Trace D).

**Kim, Y. et al. (2024), *FABLES*, COLM; arXiv:2404.01261.** Book-length faithfulness and omission annotation; motivates omission checks and claim-level sampling.

**Ru et al. (2024), *RAGChecker*, NeurIPS; arXiv:2408.08067.** Module-level RAG diagnosis; nearest diagnostic neighbor.

**Sung, Kim, and Zhang (2025), *VeriLA*, arXiv:2503.12651.** Human-centered, criteria-based agent failure verification.

**Wan et al. (2025), *Derailer-Rerailer*, Findings of ACL.** Adaptive verification for reasoning; motivates staged checks.

**Wang, G. et al. (2025), *Cer-Eval*, arXiv:2505.03814.** Certifiable, cost-efficient evaluation; motivates cost-aware selective verification.

**Sharma et al. (2024), *Towards Understanding Sycophancy in Language Models*, ICLR; arXiv:2310.13548** and **Weng, Chen, and Wang (2025), *Do as We Do, Not as You Think: The Conformity of Large Language Models*, ICLR; arXiv:2501.13381.** Preference-tracking and conformity; ground the rule that model agreement is not independent evidence.

**Shankar et al. (2024), *Who Validates the Validators?* (EvalGen), UIST; arXiv:2404.12272.** Criteria drift during grading; grounds criteria versioning.

**Grill (2024), *Constructing Capabilities: The Politics of Testing Infrastructures for Generative AI*, FAccT; doi:10.1145/3630106.3659009.** Testing infrastructure as sociotechnical and political; grounds the governance boundary in §12.4.

## 9.6 Formal methods for composition

**Lamport (1978), *Time, Clocks, and the Ordering of Events in a Distributed System*, CACM 21(7); doi:10.1145/359545.359563.** Happened-before; Definition 14.

**Goguen and Meseguer (1982), *Security Policies and Security Models*, IEEE S&P; doi:10.1109/SP.1982.10014.** Noninterference; condition N.

**Herlihy and Wing (1990), *Linearizability*, ACM TOPLAS 12(3); doi:10.1145/78969.78972.** Operation-level real-time semantics and its distinction from serializability/strict serializability; condition K must state the appropriate boundary explicitly.

**Abadi and Lamport (1991), *The Existence of Refinement Mappings*, TCS 82(2); doi:10.1016/0304-3975(91)90224-P.** Refinement mappings; condition R.

**Bauer, Leucker, and Schallhart (2011), *Runtime Verification for LTL and TLTL*, ACM TOSEM 20(4); doi:10.1145/2000799.2000800.** Three-valued finite-trace monitor semantics; the `U` verdict.

**Farach-Colton, Krapivin, and Kuszmaul (2024/2025), *Optimal Bounds for Open Addressing Without Reordering*, FOCS 2024; arXiv:2501.02305.** Expected-probe bounds in hash tables; used only as a methodological neighbor through the open-addressing probe.

## 9.7 Geometry as an order of work

**Wang and Zahl (2025), *Volume estimates for unions of convex sets, and the Kakeya set conjecture in three dimensions*, arXiv:2502.17655** and **Guth, Wang, and Zahl (2026), *A streamlined proof of the Kakeya set conjecture in* $\mathbb{R}^3$, arXiv:2601.14411.** Full dimension of Kakeya sets in $\mathbb{R}^3$. Coleman (2026b) reads the proof's architecture — sticky case, non-sticky two-scale grains, slab/plank factoring, residual — as an order for custody organization. The manuscript borrows that order and no inequality.

**Wang and Wu (2024), *Restriction estimates using decoupling theorems and two-ends Furstenberg inequalities*, arXiv:2411.08871.** Source of the two-ends vocabulary behind `TWO_ENDS`.

The authors of these geometric works are not affiliated with this research.

## 9.8 The author's former papers

**Coleman (2026a), *Index Alignment in Large Language Models: Sparse Routing, State Ledgers, and Generative Continuity*.** Defines index alignment through four consistency relations, the alignment deficit `Δ_t`, capability isolation, and sparse Mixture-of-Agents. The continuation converts each relation into a predicate family and requires a noncompensatory release policy. Scalar encodings can implement the same policy. The historical source is preserved with a linked errata record.

**Coleman (2026b), *Kakeya Grain at the Slab: Induction on Scales, Continuous Direction, and Prime-Carrying Custody under Love-Squared Coherence*.** Supplies the sticky-first order of work, HELD custody, the `Act = S·H·U` authority gate with `Act = 0 ⇏ Stop`, the `h < 1` evaluator boundary, non-compensatory grain vectors, and the object firewall between geometry and other programs.

## 9.9 Position of this work

The proposed contribution is the particular integration of four predicate families, witness paths, correction handling and partial-release decisions for exposed sparse-system indices. Noncompensatory release can be encoded as conjunction or an equivalent scalar rule. The construction applies established refinement, noninterference and monitoring ideas under stated assumptions; it does not establish a deployed checker or empirical advantage. Its geometric vocabulary supplies an organizational analogy. Positioning against proof-carrying code and agent-action certificates is necessary before making broader novelty claims.

## 9.10 Certificate and enforcement comparisons

**Necula (1997), *Proof-Carrying Code*, POPL; doi:10.1145/263699.263712.** Foundational consumer-side checking of supplied safety evidence. Compare the policy language, trusted checker and implementation assumptions; the presence of a certificate does not validate those assumptions.

**Chaki, Gurfinkel, Wallnau, and Weinstock (2009), *Assurance Cases for Proofs as Evidence*.** NASA primary record. Connects mathematical proof to actual system behavior through explicit assurance arguments about assumptions, models and implementations.

**Saltzer and Schroeder (1975), *The Protection of Information in Computer Systems*.** Primary paper copy. Least privilege, separation of privilege, fail-safe defaults and complete mediation are relevant prior foundations for the proposed authority boundary.

**Kadyrbek and Mansurova (2026), *Proof carrying agent actions enable verifiable licensing of language model tool use*, Scientific Reports; doi:10.1038/s41598-026-74900-x.** A close comparison for planner-independent action-certificate checking. The publisher reports a small certificate battery and microbenchmark with guarantees relative to trusted tool contracts. A matched stateful guardrail is a useful baseline; these results do not validate INDEX-ALIGN-CERT.

**Liu Yanglet, Wang, and Capponi (2026), *No Certificate, No Execution: Certified Traces as a Foundation for Trustworthy AI Agents*, arXiv:2605.24462.** A related proposal/certification/execution architecture at trace level. Compare conformance, proof memory, composition and trusted assumptions; this is formal/proposal background, not independent field evidence for the present system.

**Debenedetti et al. (2024), *AgentDojo*, arXiv:2406.13352.** An independently developed adversarial agent-task environment. It motivates testing useful task completion alongside security failures using held-out tasks and attacks.

---

# Claim–Evidence Matrix

Each major claim of *Boolean Index Alignment* (Manuel Coleman, 2026) is mapped to its location in the manuscript, the deep-dive section that develops it, and its cited background. The sources do not establish this construction's empirical efficacy. "Proof" means a conditional argument is written in the manuscript and annotated in the deep dive, not machine verification or implementation validation; "construction" means the object is defined in this paper; "source" means an external or prior work supports the cited point.

This is the revised claim wording. The independent review and 47-claim ledger record the original claims at commit `74c910e`; revision dispositions map those findings to these changes. All proposed LLM evaluations remain unperformed.

| # | Claim | Manuscript | Deep-dive anchor | Basis | Primary citations |
|---|---|---|---|---|---|
| 1 | Sparse-system failures are usefully analyzed as mismatches among typed discrete indices | §1.1, §3.1 | 01 §1.1–1.2 | construction + source | Coleman 2026a; Shazeer 2017; Fedus 2022; Jiang 2024; Lewis 2020; Schick 2023 |
| 2 | Registry liveness and policy permission are distinct predicates | Def. 1–2, §4.2, Trace B | 01 §1.3; 04 §4.2 | construction | Coleman 2026a Def. 1(1), Def. 3 |
| 3 | Gate weight is telemetry, not permission or quality | Def. 2, §2.3 | 01 §1.4; 04 §4.2 | source | Shazeer 2017; Lepikhin 2020; Fedus 2022; Cai W. 2024 |
| 4 | Citation lineage generalizes pointer discipline and is necessary but not sufficient for support | Def. 3, §4.4 | 01 §1.5; 04 §4.4 | construction + source | Vinyals 2015; Rashkin 2023; Gao 2023 |
| 5 | A digest alone proves neither possession nor reading; challenged spans offer sampled content evidence under stated assumptions | Def. 4, §8.5 | 01 §1.6; 06 §6.8 | construction | — (trust-boundary argument given in full) |
| 6 | Scales are declared audit levels, not model or training scales | Def. 5, §5 intro | 01 §1.7; 03 §3.1 | construction | Coleman 2026b §2.4 (order only) |
| 7 | Happened-before, not wall clocks, determines ORDER | Def. 14 | 01 §1.9; 05 §5.3 | source | Lamport 1978 |
| 8 | Semantic A/C predicates are boolean under a fixed spec; this monitor chooses T/F/U and retains diagnostic reasons | §4.1, §5.1 | 02 §2.1–2.2 | construction + source | Bauer–Leucker–Schallhart 2011 |
| 9 | Strong Kleene aggregation of sound verdicts is sound (Prop. 1) | §6.1 | 02 §2.3 | proof | — |
| 10 | PASS-only release bit; UNKNOWN and NO_VERDICT kept distinct; no hidden defaults | §4.1, App. A.1, B.1, Trace I | 02 §2.4 | construction | — |
| 11 | Release is non-compensatory; conjunction and a positive-weight scalar zero-deficit rule can encode the same policy | §2.1, §4.9 | 02 §2.5 | construction + source | Coleman 2026a Def. 2; Coleman 2026b §7.5 |
| 12 | Non-vacuity blocks release; empty denominators report n/a (Prop. 6) | Def. 27, §6.6 | 02 §2.7 | proof | — |

| # | Claim | Manuscript | Deep-dive anchor | Basis | Primary citations |
|---|---|---|---|---|---|
| 13 | Kakeya induction supplies an order of work, not an implication | §1.2, §2.2, §2.6 | 03 §3.1; 09 §9.7 | source | Wang–Zahl 2025; Guth–Wang–Zahl 2026; Coleman 2026b |
| 14 | Sound local certificates lift under R, C, N, K with Q_j corresponding to P_{M-j} | §5.4–5.7, §6.2 | 03 §3.3–3.4 | proof + source | Abadi–Lamport 1991; Goguen–Meseguer 1982; Herlihy–Wing 1990 |
| 15 | Local passes do not compose without cross-operation obligations | §6.11, Trace K | 03 §3.7 | proof (counter example) | — |
| 16 | Certificates are defeasible when any premise changes (Prop. 9) | §6.9 | 03 §3.5; 05 §5.10 | proof | — |
| 17 | Required constraints are retained on a finite DAG under stated hypotheses (Prop. 8) | §6.8 | 03 §3.8 | proof | AIce STUDY.md §§3–10 (seat contribution) |
| 18 | Stable-custody nesting uses per-registry acknowledgments, not global version comparison | §5.2 | 03 §3.6 | construction | Riemann-probe seat packet |
| 19 | A_route, A_ledger, A_gen, A_cont operationalize the four 2026a consistency relations | §4.2–4.5, App. E | 04 §4.1–4.5 | construction | Coleman 2026a Def. 1 |
| 20 | Order is not authority; append-only is not isolation | §4.3, §5.6, Trace L | 04 §4.3; 06 §6.3 | construction | Lamport 1978; Goguen–Meseguer 1982 |
| 21 | Coverage is not fidelity | §4.4, Trace C | 04 §4.4 | construction + source | Rashkin 2023; Gao 2023; Wallat 2024 |
| 22 | Exposed lineage is not proof of causal reliance | §12.1 | 04 §4.4 | source | Wallat 2024 |
| 23 | Continuity is preserved obligations across compression and handoff | §4.5, Trace N | 04 §4.5 | construction + source | Packer 2023; Park 2023; Coleman 2026a §3.3 |
| 24 | Authority requires S·H·U from channels content cannot supply; Act=0 ⇏ Stop | Def. 9 | 06 §6.3 | construction + source | Coleman 2026b §7.2 |
| 25 | Signed is not fresh; stale approvals yield U or F, never T | §6.11, Trace E | 05 §5.4 | proof (worked) | Lamport 1978 |
| 26 | Correction invalidation is reachability over the dependency graph; whole closure is rechecked | §5.8, §8.1, App. B.4 | 05 §5.6 | construction | — |
| 27 | Unaffected outputs keep their certificates under isolation (Prop. 7) | §6.7, Trace G | 05 §5.7 | proof | Goguen–Meseguer 1982 |
| 28 | Round caps bound iterations; total termination additionally requires terminating subcalls or a decision deadline | §5.8, Trace H, App. B.5 | 05 §5.9 | construction | — |
| 29 | Repetition by many agents is not authority | §8.1 | 05 §5.2 | source | Weng–Chen–Wang 2025 |
| 30 | Custody is not truth | §1.4, §5.2 | 06 §6.2 | source | Coleman 2026b §6.2–6.3 |
| 31 | Selection, gate, execution, and effect must be recorded separately | §6.8, §7.2, §10.4 | 06 §6.4 | source | AIce SCORECARD.md; PEAICE-AICE-OPTO-PROBE-001 |
| 32 | STRUCTURAL_READY is distinct from RELEASE_FULL | §5.10 | 06 §6.5 | construction | Riemann-probe seat packet |
| 33 | Five outcomes; authorized unaffected partial output survives blocked full release | §8.3–8.4, Trace M | 06 §6.6–6.7 | construction | Grok Bot seat packet; L seat |

| # | Claim | Manuscript | Deep-dive anchor | Basis | Primary citations |
|---|---|---|---|---|---|
| 34 | First failure is unique within the declared total order, which records order position and partition depth (Prop. 3) | §6.3 | 07 §7.2 | proof | — |
| 35 | The concurrent frontier is exactly the set of minimal failed DAG nodes (Prop. 4) | §6.4, Trace F | 07 §7.3 | proof | — |
| 36 | Localization is finitely computable under terminating checkers (Prop. 5) | §6.5 | 07 §7.4 | proof | — |
| 37 | Declared proper-window and directional checks distinguish selected support distributions; useful detection needs validation | Def. 6, 17, 25 | 01 §1.8; 07 §7.5 | construction + source | Wang–Wu 2024 (vocabulary); Coleman 2026b §3 |
| 38 | Position effects motivate location-stratified coverage | §12.2, Trace D | 07 §7.6 | source | Liu 2024 |
| 39 | Fourteen hand-constructed traces illustrate selected separations; coverage is not exhaustive or executed | §9 | 07 §7.8 | construction | — |
| 40 | Restricted certificate verification is O(n + #edges + cB + r + w) (Prop. 10) | §6.10, §10.2 | 08 §8.8 | proof | — |
| 41 | Verification cost does not classify search; D1–D9 | §10.3–10.4 | 08 §8.8 | construction | P-NP seat packet |
| 42 | Detection requires clean baseline and attributable named failure; nonzero exit is not detection | §11.6, Trace J | 08 §8.3 | construction | 10-agent team notes |
| 43 | Labels must be independent; model agreement is not independence | §11.2, §12.3 | 08 §8.4 | source | Sharma 2024; Weng 2025; Rashkin 2023; Kim Y. 2024 |
| 44 | Criteria must be versioned as epochs | §7.6, §12.4 | 08 §8.4 | source | Shankar 2024 |
| 45 | Evaluation includes compensatory and equivalent non-compensatory scalar policies, self-assessment, citation, coverage, global/local and human baselines | §11.7 | 08 §8.7 | construction | Ru 2024; Coleman 2026a |
| 46 | Cost of verification must be reported on all axes | §11.8 | 08 §8.8 | source | Wang G. 2025; Wan 2025; Sung 2025 |
| 47 | Certificates cannot settle who owns the specification | §12.4 | 09 §9.5; 02 §2.9 | source | Grill 2024 |

## Seat contributions reported in the original package

These paths are historical attribution, not included source files or independently verified inputs to this revision. The review did not recover the local seat packets or the 26-repository inventory. Public links are separate source pointers; reported synthetic results were not rerun.

| Seat | Material | Local path |
|---|---|---|
| Kakeyalogic | COV/ORDER/CORR/STICKY, threshold rule, geometric boundary | `/workspace/peaice/team-formalization-2026-11-05/kakeyalogic-boolean-index-r2-2026-11-05.md` |
| L | S/H/U authority, Act=0 ⇏ Stop, release boolean outcomes | `/workspace/peaice/team-formalization-2026-11-05/l-seat-boolean-index-2026-11-05.md` |
| P-NP | INDEX-ALIGN-CERT, verification bound, D1–D9 | `/workspace/peaice/team-formalization-2026-11-05/pnp-boolean-index-2026-11-05.md` |

| Seat | Material | Local path |
|---|---|---|
| Riemann probe | grains/slabs, governing acknowledgments, Aff repair, STRUCTURAL_READY vs RELEASE_FULL, eight fixtures | `/workspace/peaice/team-formalization-2026-11-05/riemann-boolean-index-2026-11-05.md` |
| AIce | DAG retention, selection/gate/execution/effect, synthetic receipts | `https://github.com/Manny536/AIce` (STUDY.md, SCORECARD.md, docs/PEAICE-AICE-OPTO-PROBE-001.md) |
| Grok Bot | diagnostic ladder, PARTIAL protocol, independent provenance, cost discipline | `/workspace/peaice/team-formalization-2026-11-05/index-alignment-grokbot-packet.md` |
| 10-agent team | R/C/N/K composition, semantic/monitor split, execution taxonomy, circularity check | `/workspace/index_alignment_team_notes.md` |
| Public Git handoff | public-repository provenance (26 commit-pinned snapshots) | `/workspace/peaice/team-formalization-2026-11-05/github-inventory-manny365-2026-11-05.md` |